16.2.2 LockModeType.OPTIMISTIC

To understand how `LockModeType.OPTIMISTIC` works, the following entities are going to be used in the upcoming test cases:

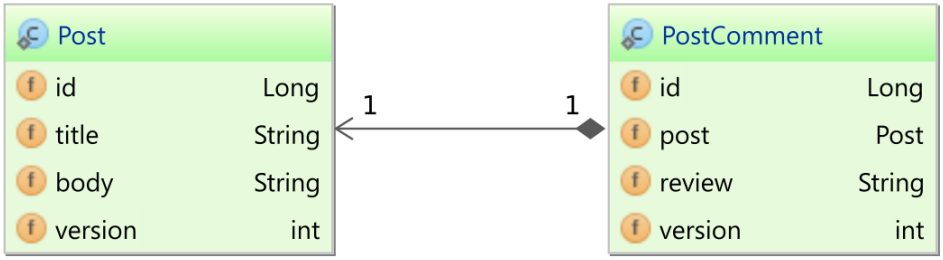

Figure 16.8: Post and PostComment entities

Once a `Post` is published, users can add `PostComment(s)` to review and share their opinions about the content of the aforementioned `Post`. Even if both entities have a `version` attribute, lost updates can be prevented at the entity level only. However, the `PostComment` entity is strictly related to the state of the `Post` entity that was used for reviewing. If a concurrent user modified the `Post` entity content, the `PostComment` might no longer be relevant.

In the following example, Alice is going to select a `Post` entity, and, while she is reviewing the `Post` entity, Bob is changing its content so that it now references the 17th chapter of the book. Alice, being unaware of the latest `Post` change, she adds a `PostComment` for the 16th chapter of the book.

```
doInJPA(entityManager -> {
    LOGGER.info("Alice loads the Post entity");
    Post post = entityManager.find(Post.class, 1L);

    executeSync(() -> {
        doInJPA(_entityManager -> {
            LOGGER.info("Bob loads the Post entity and modifies it");
            Post _post = _entityManager.find(Post.class, 1L);
            _post.setBody("Chapter 17 summary");
        });
    });

    LOGGER.info("Alice adds a PostComment to the previous Post entity version");
    PostComment comment = new PostComment();
    comment.setId(1L);
    comment.setReview("Chapter 16 is about Caching.");
    comment.setPost(post);
    entityManager.persist(comment);
});
```

When executing the test case above, Hibernate generates the following output:

```sql
-- Alice loads the Post entity
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob loads the Post entity and modifies it
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

UPDATE post
SET    body = 'Chapter 17 summary' ,
       title = 'High-Performance Java Persistence' ,
       version = 1
WHERE  id = 1 AND version = 0

-- Alice adds a PostComment review to the previous Post entity version
INSERT INTO post_comment (post_id, review, version, id)
VALUES (1, 'Chapter 16 is about Caching.', 0, 1)
```

This is still a lost update that would never happen if Alice were taking a shared lock on the `Post` entity. Unfortunately, a shared lock would compromise application scalability because Alice reviews the `Post` in the user-think time. For this reason, an optimistic lock should be acquired on the `Post` entity to ensure that the entity state hasn't change since it was first loaded.

```java
entityManager.lock(post, LockModeType.OPTIMISTIC);

LOGGER.info("Alice adds a PostComment to the previous Post entity version");
PostComment comment = new PostComment();
comment.setId(1L);
comment.setReview("Chapter 16 is about Caching.");
comment.setPost(post);
entityManager.persist(comment);
```

`LockModeType.OPTIMISTIC` does not acquire an actual lock right way, but instead it schedules a version check towards the end of the currently running transaction.

When executing the test case above while also acquiring the `LockModeType.OPTIMISTIC` on the `Post` entity, Hibernate generates the following output:

```
-- Alice adds a PostComment review to the previous Post entity version
INSERT INTO post_comment (post_id, review, version, id)
VALUES (1, 'Chapter 16 is about Caching.', 0, 1)

SELECT version FROM post WHERE id = 1

javax.persistence.OptimisticLockException: Newer version [1] of entity
[[Post#1]] found in database
```

`LockModeType.OPTIMISTIC` instructs Hibernate to check the `Post` entity version towards the end of the transaction. If the version has changed, an `OptimisticLockException` is thrown.

16.2.2.1 Inconsistency risk

Unfortunately, this kind of application-level check is always prone to inconsistencies due to bad timing. For example, after Hibernate executes the version check select statement, a concurrent transaction can simply update the `Post` entity without the first transaction noticing anything.

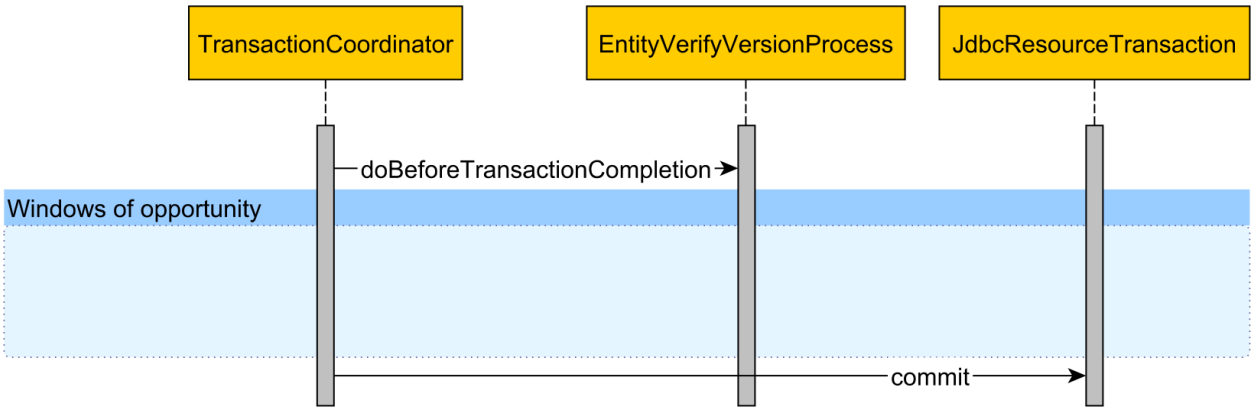

Figure 16.9: LockModeType.OPTIMISTIC window of opportunity

During that *window of opportunity*, another concurrent transaction might change the `Post` entity record before the first transaction commits its changes. To prevent such an incident, the
`LockModeType.OPTIMISTIC` should be accompanied by a shared lock acquisition:

```
entityManager.lock(post, LockModeType.OPTIMISTIC);
entityManager.lock(post, LockModeType.PESSIMISTIC_READ);
```

This way, no other concurrent transaction can change the `Post` entity until the current transaction is ended.

16.2.3 LockModeType.OPTIMISTIC_FORCE_INCREMENT

`LockModeType.OPTIMISTIC_FORCE_INCREMENT` allows incrementing the locked entity version even if the entity hasn't changed at all in the currently running Persistence Context.

The `@Version` attribute should never have a setter method because this attribute is managed automatically by Hibernate. To increment the version of a given entity, one of the two `FORCE_INCREMENT` lock strategies must be used instead.

To understand how the `LockModeType.OPTIMISTIC_FORCE_INCREMENT` strategy works, consider the following Version Control system:

Figure 16.10: Repository, Commit, and Change

The `Repository` is the root entity, and each change is represented by a `Commit` entry which, in turn, may contain one or more `Change` embeddable types.

In this particular example, the `Repository` version must be incremented with each new `Commit` being added. The `Repository` entity version is used to ensure that commits are applied sequentially, and a user is notified if a newer commit was added since she has updated her working copy.

The following example depicts the user flow for this particular Version Control system:

```
Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.OPTIMISTIC_FORCE_INCREMENT);

Commit commit = new Commit(repository);
commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));

entityManager.persist(commit);
```

When Alice executes the commit command, every file that she changes is going to be represented by a `Change` embeddable which also holds the diff between the original and the

current file content. The `Repository` entity is loaded using the `LockModeType.OPTIMISTIC_FORCE_INCREMENT` lock strategy so that its version is going to be incremented at the end of the current transaction.

Upon running the aforementioned test case, Hibernate generates the following statements:

```
SELECT r.id AS id1_2_0_, r.name AS name2_2_0_, r.version AS version3_2_0_
FROM   repository r WHERE  r.id = 1

INSERT INTO commit (repository_id, id) VALUES (1, 2)

INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '0a1,5...', 'FrontMatter.md')
INSERT INTO commit_change (commit_id, diff, path)
VALUES (2, '17c17...', 'HibernateIntro.md')

UPDATE repository SET version = 1 WHERE id = 1 AND version = 0
```

The `Repository` version is incremented before transaction completion, and, unlike `LockModeType.OPTIMISTIC`, data integrity is guaranteed by the current transaction isolation level. If the `Repository` version changed in between, the update will fail and an `OptimisticLockingException` is going to trigger a transaction rollback.

In the following example, both Alice and Bob are going to issue two commits concurrently:

```
doInJPA(entityManager -> {
    Repository repository = entityManager.find(Repository.class, 1L,
        LockModeType.OPTIMISTIC_FORCE_INCREMENT);

    executeSync(() -> {
        doInJPA(_entityManager -> {
            Repository _repository = _entityManager.find(Repository.class, 1L,
                LockModeType.OPTIMISTIC_FORCE_INCREMENT);

            Commit _commit = new Commit(_repository);
            _commit.getChanges().add(new Change("Intro.md", "0a1,2..."));
            _entityManager.persist(_commit);
        });
    });

    Commit commit = new Commit(repository);
    commit.getChanges().add(new Change("FrontMatter.md", "0a1,5..."));
    commit.getChanges().add(new Change("HibernateIntro.md", "17c17..."));
    entityManager.persist(commit);
});
```